

Universidad Nacional de La Matanza

Departamento de Ingeniería e Investigaciones Tecnológicas

Paradigmas de Programación

Grupo Delta – 2do Cuatrimestre 2026

Gradle

Integrantes

Nombre	Apellido	DNI
Pablo Alfredo	Battiston	44.893.256
Elián Ariel	Módica	42.905.229
Gustavo Gabriel	Ayala Bustos	44.109.078
Keila Mariel	Almada	46.291.918
Maria Celeste	Torres Moran	44.005.719
Lucas Leonel	Gutierrez	44.655.254

Índice

1. Introducción **3**
2. ¿Qué es Gradle y qué problema resuelve? **3**
3. Funcionamiento de Gradle: Tasks, Plugins y Build Script **4**
3.1. Tasks (Tareas) 4
3.2. Plugins y Build Script 4
4. Instalación y configuración **4**
4.1. Instalación global 4
4.2. Gradle Wrapper 5
5. Estructura de un proyecto Gradle **5**
6. Gestión de dependencias **5**
7. Ejemplo práctico **6**
8. Casos de uso e Integración con CI/CD **7**
9. Ventajas, limitaciones y comparativa: Gradle vs. Maven **7**
9.1 Ventajas 7
9.2 Limitaciones 7
9.3 Gradle vs. Maven 7
10. Conclusiones **8**
Referencias **9**

1. Introducción

El desarrollo de software moderno involucra mucho más que la escritura del código fuente. Un proyecto puede requerir la incorporación de bibliotecas externas, compilación del código, ejecución de pruebas automatizadas, generación de archivos ejecutables, empaquetado y publicación de los resultados.

Cuando estas tareas se realizan manualmente, el proceso puede volverse repetitivo, propenso a errores y difícil de mantener, especialmente cuando participan varios desarrolladores o cuando el proyecto utiliza integración y entrega continua (CI/CD). Para resolver este problema existen las denominadas herramientas de automatización de builds. Estas herramientas permiten definir y automatizar las diferentes actividades necesarias para construir un proyecto de software.

Entre las alternativas disponibles para el ecosistema Java se encuentra Gradle, una herramienta de automatización de builds de código abierto que permite gestionar el proceso de construcción de aplicaciones, incluyendo la compilación, gestión de dependencias, ejecución de pruebas y empaquetado. El objetivo de este trabajo es investigar el funcionamiento de Gradle, analizar el problema que resuelve, explicar su instalación y configuración básica en un proyecto Java, presentar un ejemplo práctico y analizar sus principales ventajas y limitaciones frente a otras alternativas, particularmente Maven.

---

2. ¿Qué es Gradle y qué problema resuelve?

Gradle es un sistema de automatización de builds de código abierto utilizado para automatizar y administrar las diferentes tareas necesarias para construir proyectos de software. Aunque suele asociarse principalmente con Java y otros lenguajes de la JVM, Gradle puede utilizarse en proyectos desarrollados con diferentes tecnologías.
Su función principal es orquestar tareas como:

- Compilación del código fuente y resolución/descarga de dependencias.
- Ejecución de pruebas automatizadas y generación de documentación.
- Empaquetado de aplicaciones, generación de artefactos y publicación de paquetes.
- Integración con herramientas de integración continua (CI/CD).

En un proyecto pequeño, un desarrollador podría realizar manualmente diferentes tareas siguiendo este flujo:

Escribir código → Compilar → Descargar librerías → Ejecutar tests → Empaquetar → Generar JAR

Sin embargo, cuando el proyecto crece, este proceso se vuelve complejo: una aplicación puede depender de decenas o cientos de bibliotecas externas y diferentes integrantes del equipo podrían utilizar distintas versiones de esas bibliotecas o herramientas. Gradle permite centralizar esta configuración en archivos del proyecto y automatizar el proceso. De esta manera, un comando como **./gradlew build** ejecuta de forma reproducible todas las tareas necesarias para construir el proyecto, lo cual resulta esencial en equipos de desarrollo y procesos de CI/CD.

3. Funcionamiento de Gradle: Tasks, Plugins y Build Script

3.1. Tasks (Tareas)

Gradle trabaja a partir de una serie de tareas (*tasks*) que representan unidades de trabajo sobre el proyecto. Estas tareas tienen dependencias entre sí; por ejemplo, la tarea **build** depende de **test** y **jar/package**, los cuales a su vez dependen de compile. Cuando se ejecuta una tarea, Gradle analiza el grafo de tareas necesarias y determina cuáles debe ejecutar, evitando realizar trabajo innecesario mediante compilación incremental cuando los resultados previos ya están actualizados. Algunos comandos fundamentales son:

- **./gradlew clean:** elimina archivos generados previamente.
- **./gradlew compileJava:** compila el código fuente Java.
- **./gradlew test:** ejecuta las pruebas unitarias automatizadas.
- **./gradlew build:** realiza el proceso completo de construcción y empaquetado.

3.2. Plugins y Build Script

La configuración de un proyecto Gradle se define mediante un *build script*, que puede escribirse utilizando Groovy (build.gradle) o Kotlin (build.gradle.kts). Dentro de este archivo se configuran tres bloques esenciales:

- **plugins:** agregan funcionalidades y tareas predefinidas a Gradle sin tener que implementarlas manualmente (por ejemplo, id 'java' incorpora las tareas para compilar, probar y empaquetar código Java; existen también plugins para Spring Boot, Android, Kotlin y calidad de código).
- **repositories:** indica en qué repositorios buscar las librerías (como mavenCentral()).
- **dependencies:** declara las bibliotecas externas requeridas por el proyecto.

---

4. Instalación y configuración

Para utilizar Gradle en un proyecto Java es necesario contar con un Java Development Kit (JDK 17 o superior) compatible con la versión de Gradle seleccionada. Si bien puede instalarse de manera global en el sistema operativo, para proyectos reales se recomienda utilizar **Gradle Wrapper**.

4.1. Instalación global

1. Descargar la distribución de Gradle (*binary-only*) desde su sitio oficial y descomprimirla.
2. Configurar la variable de entorno correspondiente y agregar el directorio **bin** de Gradle al **PATH** del sistema operativo.
3. Verificar la instalación ejecutando **gradle -v** e inicializar un proyecto mediante **gradle init**.

4.2. Gradle Wrapper

El Gradle Wrapper es un mecanismo que permite ejecutar Gradle sin necesidad de que cada desarrollador tenga instalada manualmente una versión específica de la herramienta.

El proyecto incluye los scripts gradlew (Linux/macOS), gradlew.bat (Windows) y el archivo de configuración gradle/wrapper/gradle-wrapper.properties.

- En Windows se ejecuta: **gradlew build**
- En Linux o macOS se ejecuta: **./gradlew build**

El Wrapper descarga y utiliza automáticamente la versión exacta de Gradle definida por el proyecto, garantizando consistencia entre todos los desarrolladores del equipo y en los servidores de CI/CD.

---

5. Estructura de un proyecto Gradle

Un proyecto Java básico utilizando Gradle puede tener la siguiente estructura:

```
mi-proyecto/
├── build.gradle
├── settings.gradle
├── gradlew
├── gradlew.bat
├── gradle/wrapper/
└── src/
├── main/java/
└── test/java/
```

La carpeta **src/main** contiene el código principal de la aplicación, mientras que **src/test** contiene las pruebas automatizadas. El archivo **build.gradle** contiene la configuración necesaria para construir el proyecto. Por otra parte, **settings.gradle** permite definir información general del proyecto y, en proyectos más grandes, configurar múltiples módulos.

---

6. Gestión de dependencias

Una dependencia es una biblioteca externa que nuestro proyecto necesita para funcionar (por ejemplo, **JUnit** para testing, **Spring Boot** para desarrollo de aplicaciones, **Hibernate** para persistencia o **Jackson** para procesamiento de JSON). Sin un gestor de dependencias, los desarrolladores tendrían que descargar manualmente los archivos .jar, incorporarlos al proyecto y resolver sus dependencias transitivas.

Gradle permite declarar estas dependencias de forma centralizada en el build script según el ámbito en que se requieren:



Gradle busca las bibliotecas en los repositorios configurados, las descarga automáticamente en caché y asegura que todo el equipo trabaje con las mismas versiones.

---

7. Ejemplo práctico

Para comprender el funcionamiento de Gradle en un proyecto orientado a objetos, consideremos una aplicación Java sencilla con una clase `Calculadora` que necesita realizar pruebas unitarias utilizando JUnit 5.

Archivo `build.gradle`:

```
plugins { id 'java' }
repositories { mavenCentral() }
dependencies { testImplementation 'org.junit.jupiter:junit-jupiter:5.11.0' }
test { useJUnitPlatform() }
Clase de dominio (src/main/java/Calculadora.java) y prueba unitaria  (src/test/java/CalculadoraTest.java):
// src/main/java/Calculadora.java
public class Calculadora {
 public int sumar(int a, int b) { return a + b; }
}
// src/test/java/CalculadoraTest.java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals; public class CalculadoraTest {
 @Test
 void debeSumarDosNumerosCorrectamente() {
 Calculadora calc = new Calculadora();
 assertEquals(5, calc.sumar(2, 3));
 }
}
```

Al ejecutar `./gradlew test` se corren las pruebas unitarias, y con `./gradlew build` se automatiza el flujo completo:

Código Java → Gradle (Dependencias → Compilar → Tests → Empaquetado) → Archivo JAR

---

8. Casos de uso e Integración con CI/CD

Gradle resulta especialmente beneficioso en los siguientes escenarios:

- **Proyectos Java:** permite automatizar la compilación, testing, gestión de dependencias y empaquetado.
- **Proyectos grandes y multiproyecto:** permite administrar sistemas compuestos por diferentes módulos (por ejemplo, `Sistema` dividido en `Backend`, `API`, `Servicio de usuarios` y `Servicio de pagos`), donde cada módulo puede tener sus propias dependencias y tareas.
- **Integración Continua:** puede integrarse con herramientas como Jenkins, GitHub Actions, GitLab CI y otros sistemas de automatización.
- **Proyectos con muchas dependencias y desarrollo colaborativo:** la resolución automática de librerías y el uso del Gradle Wrapper aseguran entornos idénticos entre desarrolladores.

9. Ventajas, limitaciones y comparativa: Gradle vs. Maven

9.1 Ventajas

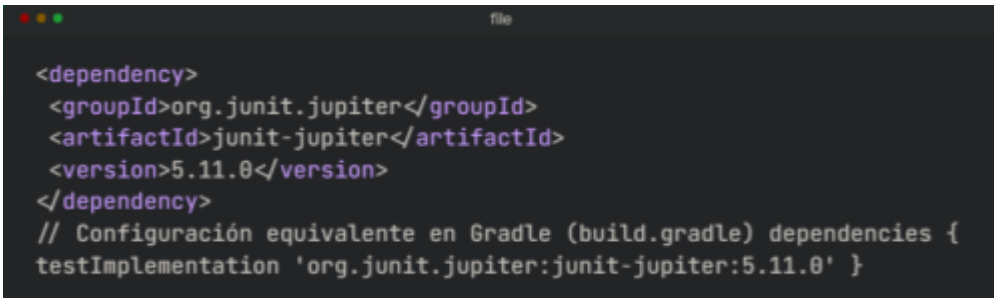
- **Automatización y gestión de dependencias:** permite automatizar el proceso de construcción del software y facilita la incorporación, actualización y resolución de bibliotecas externas.
- **Flexibilidad y rendimiento:** los build scripts permiten extender el comportamiento de Gradle según las necesidades del proyecto, mientras que la compilación incremental y el caché evitan realizar tareas innecesarias.
- **Integración con CI/CD y Gradle Wrapper:** se integra fácilmente en pipelines de automatización profesionales y garantiza que todos los desarrolladores y servidores utilicen la misma versión de Gradle.

9.2 Limitaciones

- **Curva de aprendizaje:** La flexibilidad de Gradle puede hacer que inicialmente resulte más complejo que otras herramientas.
- **Configuración:** Los proyectos con configuraciones muy personalizadas pueden generar archivos de build difíciles de mantener si no se siguen buenas prácticas. **Complejidad:** En proyectos pequeños puede ser una solución más compleja de lo necesario.

9.3 Gradle vs. Maven

Una de las principales alternativas a Gradle dentro del ecosistema Java es Apache Maven. Ambas herramientas permiten compilar, gestionar dependencias, ejecutar pruebas, empaquetar aplicaciones y publicar artefactos. Una diferencia importante es la forma de configuración. Maven utiliza principalmente archivos XML:



Característica	Gradle	Maven
Automatización de builds y gestión de dependencias	Sí	Sí
Testing e Integración con CI/CD	Sí	Sí
Lenguaje de configuración	Groovy/Kotlin	XML
Flexibilidad y personalización	Alta	Media
Curva de aprendizaje	Mayor	Menor
Convenciones	Alta, pero configurable	Muy alta

Por este motivo, la elección depende de las necesidades del proyecto, las tecnologías utilizadas y la experiencia del equipo.

---

10. Conclusiones

Gradle es una herramienta de automatización de builds que permite simplificar y estandarizar el proceso de construcción de software. Su importancia no se limita a la compilación del código, sino que abarca actividades críticas como la gestión de dependencias, ejecución de pruebas automatizadas, empaquetado y publicación de artefactos.

Una de sus características más valiosas es la posibilidad de definir el proceso de construcción mediante archivos de configuración concisos y utilizar el Gradle Wrapper para garantizar que todos los integrantes de un equipo y los servidores de integración continua utilicen exactamente la misma versión de la herramienta.

Si bien presenta una curva de aprendizaje superior a alternativas como Maven debido a su flexibilidad y capacidad de personalización, estas características representan una ventaja determinante para proyectos modulares y complejos. Finalmente, Gradle constituye un complemento esencial para la Programación Orientada a Objetos, permitiendo que el diseño de clases y paquetes desarrollado sea construido, probado y distribuido de manera automatizada, escalable y reproducible.

Referencias

Gradle Inc. (s. f.). *Gradle User Manual*. Gradle.
https://docs.gradle.org/current/userguide/userguide.html
Gradle Inc. (s. f.). *Installing Gradle*. Gradle.
https://docs.gradle.org/current/userguide/installation.html
Gradle Inc. (s. f.). *Gradle Wrapper*. Gradle.
https://docs.gradle.org/current/userguide/gradle_wrapper.html
Gradle Inc. (s. f.). *Gradle Build Tool*.
https://gradle.org/
Apache Software Foundation. (s. f.). *Maven Documentation*.
https://maven.apache.org/guides/
GeeksforGeeks. (s. f.). *What is a Build Tool?*
https://www.geeksforgeeks.org/java/what-is-a-build-tool/
Harness. (s. f.). *What is Build Automation?*
https://www.harness.io/harness-devops-academy/what-is-build-automation